

AI Model Conversation Storage

Andre Denis: Please review the web for applications that can download and store on a local drive a user's ai models conversations.

use.ai: # Research Report

Local AI Conversation Archives: Tools, Formats, and Trade-offs

Executive Summary

Two distinct jobs: Downloading an existing history and storing future chats locally are different tasks. ChatGPT's built-in export produces a ZIP, while Chatbox stores conversation history on the device and offers backup/import [2] [7]. **Action:** decide whether the priority is migration, ongoing local retention, or both.

Official baseline: ChatGPT, Claude, Gemini, and Copilot each provide a provider-specific download, but the delivery method and output vary from ZIP archives to CSV activity records [2] [9] [11] [4]. **Action:** use the first-party export before relying on a scraper or browser extension.

Cross-provider convenience: AI Exporter claims support for 12 or more major AI platforms and exports PDF, Markdown, Word, TXT, JSON, and images, while AI-chat-exporter covers ChatGPT, Claude, and Gemini with local browser handling [27] [8] [8]. **Action:** use these for selected or active chats, but treat them as privileged software that can read page content.

Durable ChatGPT archive: ChatKeeper converts an official ChatGPT export into local Markdown files, and `chatgpt-export` creates per-conversation Markdown, HTML, and JSON plus media assets [23] [25]. **Action:** use a post-processor when searchability and ownership of files matter more than staying inside a chat UI.

Local-first control: Chatbox has the clearest documented local archive workflow: JSON for backup and recovery, Markdown for reading, configurable save locations, and no automatic multi-device sync [7] [7]. **Action:** make Chatbox the leading general-purpose starting point for new conversations, with scheduled backups.

Attachment portability: Msty Studio exports a JSON file or a `.mstyconv` package containing metadata, conversation data, and an attachments folder; it recommends ZIP-style packaging when attachments reach **10 MB or more** [24]. **Action:** choose Msty when preserving media-rich sessions is important.

Self-hosting trade-off: Open WebUI exports all chats as JSON and can auto-detect ChatGPT exports, while LibreChat imports ChatGPT, Claude, and ChatbotUI files; both require a host and a backup plan [10] [13]. **Action:** use them for a shared or multi-provider server, not as a zero-administration local folder.

Security boundary: Local storage does not necessarily mean local inference: Chatbox says conversations go directly to configured AI providers, and OX Security reported a malicious extension campaign stealing ChatGPT and DeepSeek conversations associated with over **900,000** Chrome-extension downloads [7] [29]. **Action:** prefer official exports or auditable local tools, encrypt the archive, and review extension permissions.

Requirement Taxonomy: What "Local" Actually Means

The phrase "download and store" covers three different architectures. An official export is a one-time copy of a provider's account data. A page exporter reads a rendered conversation and writes a user-selected file. A local-first client stores the conversations it creates in its own local database or files. A self-hosted application stores data on the machine or server where its database and persistent volume run. The last category is local to the host, which may be a home server, office server, or cloud VPS rather than the user's laptop [5] [14].

Mode	What is captured	Where it lives	Existing hosted history	Best fit
Official provider export	Provider-defined account data and chat history	Downloaded archive on the user's drive	Yes, subject to provider scope	Full account backup
Page exporter	Current or selected rendered messages, often with media	Download folder or browser-generated file	Yes, for supported web pages	Quick readable copies
Local-first client	New chats, settings, prompts, model configuration, and sometimes attachments	Local application data folder	Usually requires import or conversion	Ongoing local retention
Self-hosted web app	Database records, uploads, metadata, and chat history	Persistent volume on the chosen host	Often supports structured imports	Teams, servers, and multi-provider use

The mechanism determines the fidelity. Official exports are closest to the provider's record but may be inconvenient or proprietary. Page exporters are convenient and readable but depend on the web interface. Local-first clients provide a durable archive from the moment they are adopted, but they do not automatically reconstruct every conversation previously held in ChatGPT, Claude, or Gemini. Self-hosted tools add migration and multi-user capabilities, but the operator becomes responsible for volumes, databases, access control, and backups.

Case study - Chatbox illustrates the difference. Chatbox stores history and API keys on the device, yet it says conversations go directly to AI service providers [7]. The observation is local retention combined with remote inference. The mechanism is that the client owns the local record while the configured provider still receives the prompt and response. The implication is that moving the archive off the provider does not by itself make a cloud API private. **Decision insight:** use the term "local archive" for storage, and separately verify where inference occurs.

Official Hosted-Service Exports: The Safest Starting Point for Existing History

Service	Documented workflow	Download or format	Important limits
ChatGPT	Profile menu -> Settings -> Data controls -> Export data -> Export	Email or SMS link to a ZIP containing chat history and other account data	May take up to 7 days ; link expires after 24 hours ; available for Free, Plus, Pro, and eligible Edu workspaces, not while logged out or for Business/Enterprise in the reviewed help article [2]
Claude	Settings -> Privacy -> Export data on the web app or Claude Desktop	Email download link containing conversation and account data	Free, Pro, and Max individual users; not iOS or Android; link expires after 24 hours ; Team or Enterprise export is controlled by the Primary Owner [9]
Gemini	Google Takeout -> select Gemini Apps data and/or Gemini Apps Activity	Google archive; an institutional guide documents a ZIP containing an HTML chat log	Activity download does not delete server data; changes between request and archive creation may be absent; creation can take hours to days [11] [6]
Microsoft Copilot	Privacy dashboard -> Copilot activity history -> Export all activity history	CSV downloaded to the device	Personal Microsoft accounts are covered; work or school accounts use separate handling; Copilot apps and Microsoft 365 activity have separate export choices [4]

The first-party route is the best default for a complete existing history because it asks the service to assemble its own record rather than scraping the visible page. ChatGPT's ZIP and Claude's conversation-data export are account-level workflows, while Copilot's documented product is an activity-history CSV, so the files are not interchangeable. Gemini's official instructions describe the Takeout selection and archive process, while the University of Michigan guide supplies the practical HTML-in-ZIP detail for its environment [11] [6].

Case study - ChatGPT to local Markdown. A user who wants ownership rather than another hosted account can request the official ChatGPT ZIP, then pass it to ChatKeeper. ChatKeeper converts the export into structured Markdown files that can be searched, backed up, opened offline, and used in Obsidian or another Markdown tool [23] [23]. The decision is conservative: acquire the provider's own export first, then transform a copy locally.

This case also exposes a limitation. ChatGPT's export contains the user's work, not the model itself [23]. A local archive therefore preserves prompts, responses, metadata, and available assets; it does not reproduce the hosted model, its weights, its future behavior, or the provider's complete application state. **Decision insight:** run the official export before account closure, migration, or a major provider change, and retain the untouched archive as the recovery source.

Dedicated Exporters and Post-Processors: More Formats, More Trust

Tool	Input scope	Local output	Strength	Caution
ChatKeeper	Official ChatGPT export from supported personal Free, Personal, and Plus plans	Structured Markdown files	Local, searchable, offline archive; not a browser scraper [23] [23]	ChatGPT-focused; encryption and access controls are not documented [23]
chatgpt-export	Official ChatGPT export ZIP	Markdown, HTML, JSON, and copied image assets in a destination directory	Command-line or Docker processing, multiple exports, readable per-chat files [25] [25]	Branch selection can affect coherence; existing same-name files may be overwritten [25]
AI Exporter	ChatGPT, Gemini, Claude, DeepSeek, Grok, Copilot, NotebookLM, Perplexity, and other claimed platforms	PDF, Markdown, Word, TXT, JSON, and images; optional Notion sync	Broad coverage and full or selected-message export [27] [27]	PDF generation is temporarily server-processed; extension and service claims require trust review [27]
AI-chat-exporter	ChatGPT, Claude, and Gemini web pages	JSON, Markdown, and PDF, with optional embedded media	Open-source, browser-agnostic local handling and selected-message export [8] [8] [8]	Shared links, some Claude artifacts, Claude code-block PDF extraction, and some PDF extraction cases are limited [8] [8]

The mechanism matters. ChatKeeper and chatgpt-export consume the official ChatGPT archive, so they are post-processors and do not need to impersonate a logged-in browser session. AI Exporter and AI-chat-exporter operate closer to the page: they are useful when a user needs one conversation, selected messages, or a common format across services. AI Exporter states that most functions run in the local browser, but its PDF path is temporarily processed on a server and then deleted [27].

Case study - fidelity versus convenience. The chatgpt-export project lets the user choose a destination directory and copies media assets, but it explicitly distinguishes a latest branch from a complete export; the complete mode can put older hidden branches into one incoherent document [25]. That is a concrete failure mode: a more complete data pull is not always a more readable narrative. The recommendation is to keep the raw ZIP, generate a human-readable archive in latest mode, and retain the complete output only for forensic or machine-processing needs.

The browser option has a different failure mode. The AI-chat-exporter documentation says complex Claude artifact or preview extraction can fail, and shared links are not supported [8]. **Decision insight:** use post-processors for organization and format conversion, but use provider exports for completeness; never make a third-party extension the only copy of a sensitive archive.

Local-First Desktop Clients: Best for Future Conversations

Application	Local-storage evidence	Export or backup evidence	Model/provider scope	Practical trade-off
Chatbox	History, settings, prompts, model configuration, and API keys are stored on-device; platform paths are documented [7]	JSON backup/import and Markdown export; desktop save location is selectable [7]	Homepage lists OpenAI, Gemini, Anthropic, DeepSeek, xAI, Mistral, OpenRouter, Azure AI, AWS Bedrock, and others [3]	No automatic multi-device sync; lost local data cannot be recovered from Chatbox servers [7]
Jan	Local, privacy-first app; chat archive uses messages.jsonl and thread.json under threads/ [18] [15]	Data-folder path can be customized; the reviewed docs expose raw archive files rather than a separate export wizard [15]	Open-source replacement for Claude and ChatGPT; the reviewed pages do not establish complete provider coverage [18]	Strong file-level control, but more manual handling
AnythingLLM Desktop	Product page says model, documents, and chats are stored locally; no account is needed and desktop builds support macOS, Windows, and Linux [19]	CSV, JSON, Alpaca JSON, and OpenAI fine-tune JSONL export; the chat-log page says export appears once at least 10 logs exist [12]	Workspace and document-chat focus; the cited local-storage page does not enumerate every provider	Good for RAG/workspaces; confirm the chosen model endpoint before assuming inference is local
Msty Studio	Privacy-first platform for local and online models; Studio Web stores data in browser local storage [17]	.JSON or .mstyconv ; JSON embeds attachments, while the package includes metadata, split data, and an attachments folder [24]	Local and online models, with providers added through the Studio workflow [17]	Strong media portability; browser-local storage still needs deliberate backup
Cherry Studio	Supports fully local usage scenarios and says local models can avoid data-leakage risk; it also connects mainstream providers [20] [20]	Complete or partial conversation export such as Markdown and Word; local, WebDAV, and scheduled backup options are documented [20] [20]	Unified access to OpenAI, Gemini, Anthropic, Azure, and compatible third-party standards [20]	Broad model comparison, but verify the exact desktop data path and export schema before standardizing on it

Case study - Chatbox as an ongoing archive. The user experience is unusually explicit: Settings -> Data Management -> Export Data, choose a location and format, and use JSON for recovery or Markdown for reading [7]. The mechanism is an application-owned archive rather than a provider-side export. The outcome is a practical local workflow across desktop platforms, with Android exports placed under `Documents/chatbox_ai_exports` [7].

The trade-off is operational responsibility. Chatbox says each device stores its data independently, and manual movement requires export, transfer, and import [7]. It also says the server cannot recover lost local chats [7]. **Decision insight:** Chatbox is the strongest general local-first recommendation in this review, provided the user treats JSON backups like irreplaceable data.

Jan, LM Studio, and GPT4All are better understood as local-model chat environments than as universal hosted-history downloaders. Jan exposes a transparent JSON/JSONL archive [15]. LM Studio documents threads and folders for local LLM chats [16]. GPT4All documents chats with models running locally and a chat-history view [26]. These are attractive when the model itself runs on the computer, but the reviewed documentation does not show the same cross-provider import and export workflow as Chatbox or Open WebUI. **Decision insight:** choose them for local inference first, archive conversion second.

Self-Hosted Web Applications: Local to the Host, Not Necessarily to the Laptop

Application	Conversation movement	Storage and scope	Best use
Open WebUI	Exports all chats to JSON with messages, metadata, timestamps, and structure; imports Open WebUI, ChatGPT, and expected custom JSON [10]	Docker data must be persisted; the documented data store includes <code>/app/backend/data</code> , <code>webui.db</code> , uploads, and vector data [5]	A structured, self-hosted archive with ChatGPT migration
LibreChat	Exports screenshots, Markdown, text, and JSON; imports ChatGPT, Claude, and ChatbotUI v1 archives [14] [13]	Uses MongoDB to store and retrieve conversation histories; the deployment can be local or cloud depending on how it is installed [28] [14]	Multi-provider, multi-user, or server-based operation

Open WebUI offers the clearest migration semantics. It recognizes a ChatGPT export when the first object contains a `mapping` key and converts it during import [10]. It does not provide a built-in converter for Claude or Gemini; those exports must be transformed into the expected message-tree structure [10]. Re-importing creates new IDs and can create duplicates [10].

Case study - Open WebUI and Docker persistence. The application can appear to work while its container is disposable. The backup documentation says containers are ephemeral unless data is persisted on the host, and gives `/app/backend/data` as the mount target; it identifies `webui.db`, `uploads/`, and `vector_db/` as persistent components [5]. The mechanism is a database-backed server rather than a loose folder of Markdown files. The implication is that "self-hosted" only becomes durable after volume mapping and backup testing. **Decision insight:** use Open WebUI when structured ChatGPT migration and an administrator-controlled server matter, and back up the database and data directory before upgrades.

LibreChat has a different balance. It supports a wide range of local and remote providers and can import ChatGPT and Claude archives directly after extracting `conversations.json` [14] [13]. MongoDB is central to its history model [28], so it is more powerful for shared deployments but more operationally demanding than a single-user Markdown archive. **Decision insight:** choose LibreChat for a provider hub or team server; choose a desktop local-first client when the requirement is simply "save my chats on this drive."

Privacy, Security, Recovery, and Portability

Risk or tension	Evidence from the reviewed applications	Control to apply
Local storage versus cloud inference	Chatbox stores chats and keys locally but says conversations go directly to AI service providers [7]; Msty supports both local and online models [17]	Select a local model or endpoint when data must not leave the device; otherwise classify the archive as local storage of cloud-generated content
Extension privilege	OX Security reported a campaign stealing ChatGPT and DeepSeek conversations associated with over 900,000 Chrome-extension downloads [29]	Prefer official exports or auditable open-source tools; install extensions in a separate browser profile and review permissions
Temporary server processing	AI Exporter says most export functions run in the browser, but PDF generation is temporarily processed on its server and then deleted [27]	Use Markdown, TXT, JSON, or image output when possible; do not send regulated or confidential content through an unreviewed conversion service
Lost local data	Chatbox says it cannot recover chats from its servers because history is only on local devices [7]	Automate versioned backups to an encrypted second drive or controlled storage; test restoration
Disposable containers	Open WebUI says Docker containers are ephemeral and identifies persistent volume and database files for backup [5]	Map <code>/app/backend/data</code> , back up <code>webui.db</code> , and test a clean restore before upgrades
Format mismatch and duplicates	Open WebUI requires a JSON array and creates new IDs on import; repeated imports create duplicates [10]	Preserve the original export, validate one sample, import into a disposable workspace, and record the archive checksum

The central observation is that "local" is a storage property, not a complete privacy guarantee. The mechanism may still involve a cloud model provider, a server-side PDF conversion, or a browser extension with page access. The implication is that a local Markdown file can be safer to retain than a hosted history while still containing sensitive material and API configuration. The reviewed ChatKeeper page, for example, does not document encryption, authentication, or access controls [23].

Case study - browser convenience versus supply-chain risk. A page exporter can make a difficult workflow one click, but the OX report shows why the same browser privilege is a material risk: a campaign was reported to steal conversations from popular AI sites at large install scale [29]. The outcome is not that every exporter is malicious; it is that the user must distinguish the tool's advertised local handling from the trustworthiness of its distribution and update path. **Decision insight:** for sensitive archives, start with a first-party export or a locally run open-source post-processor, then protect the resulting files with full-disk encryption and least-privilege access.

Portability also has layers. Markdown is readable across tools; JSON is better for structured recovery; Msty's `.mstyconv` is richer for its own attachments but application-specific [24]. Claude's help page says exported data cannot be imported into another personal Claude account [9], which is a reminder that an export file is not automatically a migration contract. **Decision insight:** keep both the untouched provider archive and a human-readable derivative whenever storage capacity permits.

Decision Matrix and Implementation Workflow

--	--	--	--

Need	First choice	Why it fits	First step
Download a complete existing ChatGPT history	OpenAI export, then ChatKeeper or chatgpt-export	First-party ZIP plus local Markdown/HTML/JSON conversion [2] [23] [25]	Request the export and preserve the raw ZIP
Download one or several current web chats	AI Exporter or AI-chat-exporter	Broad service coverage or ChatGPT/Claude/Gemini support with multiple local formats [27] [8]	Verify the extension source and export a non-sensitive test chat
Start a general multi-provider local archive	Chatbox	Explicit local history, JSON restore, Markdown output, and broad provider list [7] [3]	Set a custom backup destination and schedule manual exports
Preserve conversations with large attachments	Msty Studio	JSON embeds attachments and .mstyconv packages them; the product recommends the package for media-heavy sessions [24]	Export one test conversation and confirm attachment restoration
Build a local document/RAG workspace	AnythingLLM Desktop	The desktop product says model, documents, and chats are local; chat logs export to several structured formats [19] [12]	Configure the storage location and export after the documented log threshold
Expose several providers to a server or team	LibreChat	Self-hosted provider hub with JSON/Markdown/text export and ChatGPT/Claude imports [14] [14] [13]	Deploy on a host with a persistent MongoDB backup
Migrate ChatGPT history into a self-hosted UI	Open WebUI	ChatGPT JSON is detected and converted; all chats can be exported as structured JSON [10]	Persist /app/backend/data and test one import
Run models locally and retain local threads	Jan, LM Studio, or GPT4All	Their documentation centers on local models and on-device chat history [18] [16] [26]	Confirm the data folder and copy a sample archive before upgrading

A reliable implementation is a five-step pipeline. First, request the official export from every hosted service that contains historical chats. Second, store the untouched ZIP, Takeout archive, or CSV in an encrypted archive directory. Third, generate readable Markdown or HTML derivatives for search and a structured JSON copy for future conversion. Fourth, if adopting Chatbox, Msty, Open WebUI, or LibreChat, import a small sample and verify messages, timestamps, attachments, and code blocks. Fifth, schedule a recurring backup and document which provider, account, date, and conversion tool produced each file.

The recommendation is intentionally layered rather than centered on one product. Chatbox is the

The recommendation is intentionally layered rather than centered on one product. Chatbox is the best documented single-user local-first default; ChatKeeper and `chatgpt-export` are the cleanest ChatGPT post-processors; Msty is strongest for portable attachments; Open WebUI and LibreChat are stronger when a server or multi-provider migration is the goal. **Decision insight:** do not replace the official export with a new application until the raw export has been secured and tested.

Synthesis

Strategy and major examples	Mechanism	Scope	Trade-off	Evidence and time horizon
First-party exports: ChatGPT, Claude, Gemini, Copilot	Provider assembles account or activity data	Broadest access to that provider's own record	Provider-specific formats, delays, expiry, and account limits	Strongest evidence for existing history; one-time or periodic
Post-processors: ChatKeeper and <code>chatgpt-export</code>	Read an official archive and create local files	ChatGPT-focused, with Markdown/HTML/JSON and media options	High fidelity to the source archive, but not a universal provider solution	Strong local-file evidence; repeat after each official export
Page exporters: AI Exporter and AI-chat-exporter	Read a supported web UI and save selected or current content	Broader provider coverage and convenient formats	UI breakage, extension privilege, and in one case temporary server PDF processing	Useful for immediate capture; continuous use depends on maintenance and trust
Local-first clients: Chatbox, Jan, AnythingLLM, Msty, Cherry	Store new chats in an application archive on a device	Ongoing conversations across local, online, or configured providers	Does not automatically reconstruct every old hosted history; backup is the user's responsibility	Best for future chats; continuous archive from adoption onward
Self-hosted platforms: Open WebUI and LibreChat	Store structured histories in a persistent database on a chosen host	Multi-provider, migration, and multi-user workflows	Volume/database administration, access control, and restore burden	Best for a managed server; durable only when persistence is configured

The non-obvious tension is between convenience and evidentiary control. Official exports are

authoritative but slow and provider-bounded. Page exporters are broad and fast but depend on page structure and elevated browser access. Local-first clients minimize dependence on a provider's history UI, yet Chatbox and Msty demonstrate that local storage can coexist with cloud inference [7] [17]. Self-hosted tools improve migration and collaboration, but their localness is defined by the host: Open WebUI's Docker volume and database must be backed up [5], and LibreChat's history model depends on MongoDB [28].

The products also diverge in what they optimize. Chatbox optimizes documented single-user backup and restore. Jan exposes transparent JSON/JSONL files. AnythingLLM emphasizes local documents and workspaces. Msty emphasizes an importable conversation package with attachments. Cherry emphasizes model comparison, provider breadth, and backup choices. ChatKeeper and `chatgpt-export` optimize ChatGPT archive conversion, while AI Exporter and AI-chat-exporter optimize cross-provider page capture. Open WebUI emphasizes structured ChatGPT migration, and LibreChat emphasizes a broad self-hosted provider hub [7] [15] [12] [24] [20] [23] [25] [10] [13].

The practical conclusion is a two-layer archive: provider exports as immutable source material, plus a local-first or self-hosted application for future work. Use Markdown for durable reading, JSON for recovery and transformation, and application packages only when their attachment fidelity is valuable. Encrypt the files, test one restore, and treat browser extensions and cloud endpoints as separate trust decisions. **Final decision:** for most individual users, start with official exports plus ChatKeeper for legacy ChatGPT data and Chatbox for new multi-provider conversations; add Msty for media-heavy work, and move to Open WebUI or LibreChat only when server control or migration breadth justifies the operational cost.

References

Export Chat - Msty Docs. <https://docs.msty.app/features/export-chat>

Exporting your ChatGPT history and data | OpenAI Help Center.

<http://help.openai.com/en/articles/7260999-exporting-your-chatgpt-history-and-data>

Chatbox AI. <https://chatboxai.app/en>

Manage your Copilot activity history in the privacy dashboard | Microsoft Support.

<https://support.microsoft.com/en-us/privacy/manage-your-copilot-activity-history-in-the-privacy-dashboard>

Backups / Open WebUI. <https://docs.openwebui.com/tutorials/maintenance/backups/>

Article - Google: Export Gemini Chat ...

Data Storage | Chatbox AI User Guide. <https://chatboxai.app/en/guide/faq/data-storage>

GitHub - TheBluCoder/AI-chat-exporter: Export conversations from AI platforms (Gemini, Claude, ChatGPT). Download chats as JSON, Markdown, or PDF with full media preservation · GitHub.

<https://github.com/TheBluCoder/AI-chat-exporter>

Export your Claude data | Anthropic Help Center. <http://support.claude.com/en/articles/9450526->

export-your-claude-data

Import & Export / Open WebUI. <https://docs.openwebui.com/features/chat-conversations/data-controls/import-export/>

Download your Gemini Apps data - Gemini Apps Help.

<https://support.google.com/gemini/answer/16920332?hl=en>

Workspace Chat Logs ~ AnythingLLM. <https://docs.anythingllm.com/features/chat-logs>

Import Conversations | LibreChat. https://www.librechat.ai/docs/features/import_convos

GitHub - danny-avila/LibreChat: Enhanced ChatGPT Clone: Features Agents, MCP, Skills, DeepSeek, Anthropic, AWS, OpenAI, Responses API, Azure, Groq, o1, GPT-5, Mistral, OpenRouter, Vertex AI, Gemini, Artifacts, AI model switching, message search, Code Interpreter, langchain, DALL-E-3, OpenAPI Actions, Functions, Secure Multi-User Auth, Presets, open-source for self-hosting. Active · GitHub. <https://github.com/danny-avila/LibreChat>

Jan Data Folder. <https://www.jan.ai/docs/desktop/data-folder>

Manage chats. <https://lmstudio.ai/docs/app/basics/chat>

Welcome to Msty Studio - Msty Studio Docs. <http://docs.msty.ai/studio/getting-started>

Overview. <https://www.jan.ai/docs>

AnythingLLM — On-device AI for productivity | Local & Private. <http://anythingllm.com/>

Project introduction. <https://docs.cherryai.com.cn/docs/en-us>

General Desktop Information ~ AnythingLLM. <https://docs.anythingllm.com/installation-desktop/storage>

Chat Auto Exporter. <https://lmstudio.ai/prompts/ai/chat-auto-exporter>

ChatKeeper: Export and Keep Your Entire ChatGPT History as Local Markdown.

<https://www.martiansoftware.com/chatkeeper/keep-your-chatgpt-history/>

Conversation Export and Import - Msty Studio Docs.

<https://docs.msty.ai/studio/conversations/export-import>

GitHub - jamesmoore/chatgpt-export: Extracts chats from a ChatGPT export file into individual markdown and formatted html files including media assets.

<https://github.com/jamesmoore/chatgpt-export>

Gpt4All Desktop. https://docs.gpt4all.io/gpt4all_desktop/chats.html

AI Exporter - ChatGPT to PDF & Gemini to PDF. <http://saveai.net/>

MongoDB. https://www.librechat.ai/docs/user_guides/mongodb

Malicious Chrome Extensions Steal ChatGPT Conversations.

<https://www.ox.security/blog/malicious-chrome-extensions-steal-chatgpt-deepseek-conversations/>